STYLESENSE

Fashion Forward Forecasting

Machine Learning Pipeline for Product Recommendation Prediction

Data Science Project Report

May 2026

1. Executive Summary

StyleSense, a rapidly growing online women's clothing retailer, faces a critical operational challenge: a large and growing backlog of customer product reviews with incomplete recommendation data. While customers actively engage with the platform by writing detailed review text, they frequently omit the structured recommendation indicator — a binary field that signals whether they endorse the product to other shoppers. This gap directly impairs the retailer's ability to leverage customer sentiment at scale, undermining product ranking, merchandising decisions, and personalisation capabilities.

This report presents a complete, production-ready machine learning pipeline designed to address this challenge. The pipeline ingests raw review data — encompassing free-form text, customer demographic information, and categorical product metadata — and produces a reliable prediction of whether a customer would recommend a product. The system achieves **85% accuracy** on held-out test data, with a weighted F1-score of **86%**, demonstrating strong generalisation across both the majority (recommended) and minority (not recommended) classes.

The solution is built on a modular scikit-learn pipeline that integrates three distinct preprocessing pathways: **StandardScaler** for numerical features, **OneHotEncoder** for categorical features, and a **spaCy en_core_web_sm**-powered NLP chain for text features. The NLP chain applies statistical lemmatisation, part-of-speech (POS) filtering, dependency-parse negation detection, named entity recognition (NER), and TF-IDF vectorisation. A logistic regression classifier with class-weight balancing completes the pipeline. Hyperparameters were optimised using GridSearchCV with stratified 3-fold cross-validation.

The project also delivers an interactive **Streamlit dashboard** that loads the trained pipeline from a serialised pickle file and provides four live views: dataset exploration, real model metrics, feature importance analysis, and a live prediction interface that runs the actual pipeline on any review entered by the user. All three standout sections specified in the project rubric are fully implemented: advanced NLP with *en_core_web_sm*, feature visualisations, and the data dashboard.

2. Problem Statement

StyleSense operates in a highly competitive online retail environment where customer-generated content is a primary driver of purchasing decisions. The platform collects structured and unstructured feedback through a review system that captures product ratings, free-text descriptions, and a binary recommendation indicator (*Recommended IND*). However, a significant portion of submitted reviews contain the free-text component but are missing the recommendation field, either because the customer did not complete the form or because the data was lost during system migration.

This incompleteness creates a compounding business problem. Without a recommendation signal, the platform cannot accurately rank products, cannot identify which items are generating customer dissatisfaction, and cannot feed reliable signals into downstream personalisation or merchandising algorithms. Manual review and labelling of the backlog is infeasible at scale given the volume of incoming reviews and the associated labour cost.

The core machine learning problem is therefore a **supervised binary classification task**: given a review that contains text, demographic, and product category data, predict whether the customer would assign a positive recommendation (class 1) or not (class 0). This is complicated by three factors. First, the dataset exhibits **class imbalance** — approximately 82% of reviews are positive — which risks a naive classifier simply predicting the majority class. Second, the features span three fundamentally different data types (numerical, categorical, and free text), requiring a heterogeneous preprocessing strategy. Third, text data introduces high dimensionality through vocabulary variation, negation ambiguity, and domain-specific language (e.g., clothing terminology, fit descriptions, material references) that standard bag-of-words approaches handle poorly without linguistic normalisation.

The solution must not only achieve strong predictive performance but must also be structured as a proper sklearn pipeline so that preprocessing steps are learned exclusively from training data and can be safely applied to unseen reviews at inference time — preventing data leakage and ensuring deployment readiness.

3. Dataset

The dataset used in this project is *reviews.csv*, a curated collection of **18,442 anonymized women's clothing product reviews** sourced from StyleSense's internal review system. The data has been pre-cleaned and contains no missing values, enabling immediate use in model training without imputation steps. Each record represents a single customer review and contains eight input features alongside the binary prediction target.

3.1 Feature Summary

Column	Type	Description
Clothing ID	Integer (Categorical)	Anonymized product identifier
Age	Integer (Numerical)	Customer age in years
Title	String (Text)	Short headline of the review
Review Text	String (Text)	Full body of the review
Positive Feedback Count	Integer (Numerical)	Number of customers who found the review helpful
Division Name	String (Categorical)	High-level product division (e.g., General, Petite)
Department Name	String (Categorical)	Product department (e.g., Tops, Dresses, Bottoms)
Class Name	String (Categorical)	Product class (e.g., Knits, Blouses, Dresses)
Recommended IND	Binary (0/1)	Target: 1 = Recommended, 0 = Not Recommended

3.2 Target Distribution and Class Imbalance

The target variable is significantly imbalanced. Of the 18,442 reviews, **15,053 (81.6%)** are positive (Recommended = 1) and **3,389 (18.4%)** are negative (Recommended = 0). This 4.4:1 imbalance ratio means that a naive majority-class classifier would achieve ~82% accuracy without learning any meaningful pattern. The model addresses this through **class_weight='balanced'** in the logistic regression classifier, which inversely weights

each class by its frequency and ensures the model is penalised equally for errors on both classes.

3.3 Key Exploratory Findings

- Recommendation rates vary across departments: Bottoms (85.3%) and Jackets (84.1%) have the highest rates, while Trend (75.7%) has the lowest, suggesting category is a useful signal.
- Older customers (60–70 age band) recommend products at a higher rate (~86%) than younger customers (25–35 age band: ~79–80%), indicating age is informative.
- Recommended reviews tend to be longer and contain more exclamation marks and positive adjectives, while non-recommended reviews contain more negation patterns and return-related vocabulary.
- Positive Feedback Count and Clothing ID show low direct correlation with the target, but contribute useful context when combined with text signals.

4. Solution Statement

The solution is an end-to-end scikit-learn **Pipeline** consisting of a custom **FullFeatureUnion** transformer followed by a **Logistic Regression** classifier. The pipeline encapsulates all preprocessing and feature engineering steps so that no transformation is computed outside of the pipeline boundary — ensuring that the same operations applied during training are automatically applied during inference without any risk of data leakage.

4.1 Feature Engineering Strategy

The pipeline handles three fundamentally different data types within a single *fit/transform* interface:

- **Numerical features** (Age, Positive Feedback Count, Clothing ID): Scaled to zero mean and unit variance using **StandardScaler**. This prevents features with larger magnitudes from dominating the logistic regression weight updates.

- **Categorical features** (Division Name, Department Name, Class Name): Encoded using **OneHotEncoder** with handle_unknown='ignore'. This converts nominal categories into binary indicator columns while gracefully handling unseen categories at inference time.
- **Text features** (Title + Review Text): Processed through a two-stage NLP pipeline — a spaCy-powered normaliser followed by TF-IDF vectorisation — plus a 20-feature handcrafted NLP vector described in Section 5.

4.2 Text Preprocessing with spaCy en_core_web_sm

The text preprocessing pipeline uses the **en_core_web_sm** pretrained model to perform:

- Linguistic tokenisation — accurate handling of contractions and punctuation
- Lemmatisation via token.lemma_ — collapses inflected forms (dresses→dress, recommended→recommend), reducing TF-IDF sparsity
- Stop word removal using spaCy's built-in 326-word English stop list (token.is_stop)
- POS filtering — retains only NOUN, VERB, ADJ, ADV tokens, discarding determiners, prepositions, and pronouns that carry little sentiment signal

4.3 Model Selection and Class Imbalance

Logistic Regression was selected as the classifier for its interpretability (coefficients directly show feature importance), computational efficiency on sparse TF-IDF matrices, strong baseline performance on text classification tasks, and native support for probability estimates. The L2 regularisation strength (C) was treated as a hyperparameter. Class imbalance was addressed through **class_weight='balanced'**, which weights each sample inversely proportional to its class frequency.

4.4 Hyperparameter Optimisation

GridSearchCV with 3-fold stratified cross-validation was used to tune three hyperparameters simultaneously: TF-IDF vocabulary size (300 or 500 features), n-gram range (unigrams only or unigrams + bigrams), and regularisation strength C (1.0 or 10.0). The best configuration (C=10, 500 features, bigrams) was automatically refit on the full training set before evaluation on the held-out test set.

5. Architecture

The system architecture is modular and consists of four layers: a data layer, a feature engineering layer, a model layer, and an application layer. All components are designed to be portable — the trained pipeline is serialised to a single pickle file that can be loaded by any Python application without re-training.

5.1 System Components

Component	File	Purpose
Data Layer	data/reviews.csv	Raw input: 18,442 reviews with 8 features + target
Pipeline Classes	pipeline_model.py	All custom sklearn transformers and the load_nlp() factory
NLP Model	en_core_web_sm (spaCy)	Pretrained statistical NLP: POS tagger, parser, lemmatiser, NER
Training Notebook	pipeline_project.ipynb	EDA, pipeline construction, GridSearchCV, evaluation
Trained Artefact	pipeline_payload.pkl	Serialised pipeline + test data for the dashboard
Dashboard	dashboard.py	Streamlit app: loads pkl, shows metrics, runs live predictions

5.2 FullFeatureUnion — Single-Pass Design

The core transformer class **FullFeatureUnion** is engineered to perform a **single nlp.pipe() call** per fit/transform invocation, simultaneously extracting both the normalised text for TF-IDF and the 20-element numeric NLP feature vector. This halves spaCy processing time compared to running two separate passes. The output is a horizontally concatenated dense numpy matrix:

> Output = [structured (23 cols) | TF-IDF (up to 500 cols) | spaCy NLP (20 cols)] → 543 total features

5.3 spaCy NLP Feature Vector (20 Features)

#	Feature	spaCy Component	Signal Captured
0–5	char_count, word_count, sent_count, excl_count, quest_count, avg_word_length	Tokeniser	Review depth, punctuation emotion signals
6–11	adj_count, adv_count, verb_count, noun_count, adj_ratio, adv_ratio	Statistical POS tagger	Grammatical composition; positive reviews are adjective-rich
12	negation_count	Dependency parser	Count of neg arcs: 'not recommend', 'doesn't fit'
13	neg_adj_count	Dependency parser	'not flattering' treated as negative, not positive
14	unique_lemma_ratio	Lemmatiser	Vocabulary richness on lemmatised content words
15–17	clothing_ents, fit_ents, material_ents	NER (EntityRuler+stat.)	Domain entity counts (CLOTHING, FIT, MATERIAL)
18	sentiment_pos_ents	NER (EntityRuler)	Named positive sentiment word count

#	Feature	spaCy Component	Signal Captured
19	net_sentiment_ents	NER (EntityRuler)	SENTIMENT_POS minus SENTIMENT_NEG

#	Feature	spaCy Component	Signal Captured
0–5	char_count, word_count, sent_count, excl_count, quest_count, avg_word_length	Tokeniser	Review depth, punctuation emotion signals
6–11	adj_count, adv_count, verb_count, noun_count, adj_ratio, adv_ratio	Statistical POS tagger	Grammatical composition — positive reviews are adjective-rich
12	negation_count	Dependency parser	Number of 'neg' dependency arcs
13	neg_adj_count	Dependency parser	Negated adjectives — 'not flattering' ≠ 'flattering'
14	unique_lemma_ratio	Lemmatiser	Vocabulary richness on lemmatised content words
15–17	clothing_ents, fit_ents, material_ents	NER (EntityRuler + stat.)	Domain entity counts

#	Feature	spaCy Component	Signal Captured
18–19	sentiment_pos_ents, net_sentiment_ents	NER (EntityRuler)	Named entity sentiment balance

5.4 Pipeline Flow

1. Raw review DataFrame passed to pipeline.fit(X_train, y_train)
2. FullFeatureUnion.fit(): fits ColumnTransformer on structured columns; runs nlp.pipe() once to fit TfidfVectorizer on lemmatised text
3. FullFeatureUnion.transform(): single nlp.pipe() pass produces normalised text AND 20-element NLP feature vector simultaneously; ColumnTransformer applies scalers and encoders; outputs are horizontally stacked
4. LogisticRegression.fit(): trains on the 543-feature matrix with class_weight='balanced'
5. GridSearchCV refits best estimator on full training set (refit=True)
6. Evaluation on held-out 10% test set (stratified split, random_state=27)

6. Dashboard

The interactive dashboard (**dashboard.py**) is a **Streamlit** application that loads the trained pipeline from *pipeline_payload.pkl* and provides four fully connected views. Unlike a static HTML report, every metric, chart, and prediction in the dashboard is computed directly from the *live pipeline object* — there are no hardcoded numbers. The dashboard is launched with *streamlit run dashboard.py* from the project directory.

6.1 Tab 1 — Dataset Overview

The Dataset Overview tab provides a data-driven introduction to the review corpus. It displays five headline KPI metrics (total reviews, feature count, overall recommendation

rate, number of departments, and number of product classes) computed directly from the loaded DataFrame. Four visualisations are provided:

- Target Class Distribution — a dual-panel bar + pie chart showing the 81.6% / 18.4% class split, with an annotation explaining the class_weight='balanced' treatment applied in the model
- Recommendation Rate by Age Group — a bar chart showing recommendation rates across five-year age bins, revealing that older customers (60–70) are more likely to recommend products than customers aged 25–35
- Recommendation Rate by Department — a horizontal bar chart comparing six product departments, with the overall average shown as a reference line
- Review Length Distribution — overlapping histograms comparing word count distributions for recommended versus not-recommended reviews, demonstrating that longer, more detailed reviews are associated with positive recommendations

A raw data sample table (10 randomly sampled rows) is displayed at the bottom of this tab, allowing the user to inspect actual review content.

6.2 Tab 2 — Model Performance

The Model Performance tab computes all metrics in real time from *y_test* and *y_pred* stored in the payload. Four metric cards display accuracy (85%), weighted precision (88%), weighted recall (85%), and weighted F1-score (86%). A **ConfusionMatrixDisplay** plot from scikit-learn shows the breakdown of true negatives, true positives, false positives, and false negatives on the 1,845-sample test set. A grouped bar chart displays precision, recall, and F1 per class, highlighting that while performance on the majority class (Recommended) is strong (F1 = 0.90), the minority class (Not Recommended) achieves a respectable F1 of 0.67 given the 4.4:1 imbalance.

A prediction confidence histogram shows the distribution of predicted probabilities for the positive class, split by actual label — allowing the user to visually assess model calibration and see that the model assigns high confidence (> 0.8) to the vast majority of correctly classified positive reviews.

6.3 Tab 3 — Feature Analysis

The Feature Analysis tab extracts logistic regression coefficients directly from *pipeline.named_steps['clf'].coef_[0]* and joins them with feature names obtained from *get_feature_names(pipeline)*. A slider allows the user to control how many top features to display on each side. A radio selector filters features by type: All, TF-IDF words, spaCy NLP features, or Structured features.

Two horizontal bar charts show the top positive (pushes toward Recommended) and top negative (pushes toward Not Recommended) features. High-weight positive words include 'love', 'perfect', 'flattering', 'comfortable', and 'gorgeous'. High-weight negative features include 'unflattering', 'disappointed', 'unfortunately', 'return', and 'tight'. A dedicated spaCy NLP feature coefficient chart shows the model weight assigned to each of the 20 handcrafted features, with *net_sentiment_ents* and *adj_count* emerging as strong positive signals, and *negation_count* and *neg_adj_count* as strong negative signals — validating the utility of the dependency-parse negation features.

6.4 Tab 4 — Live Prediction

The Live Prediction tab allows any user to enter a review through a web form and receive a real-time prediction from the trained pipeline. The form accepts: review title, review text, customer age, helpful vote count, department, division, and class name. On clicking Predict, the dashboard constructs a single-row DataFrame matching the exact training schema and calls *pipeline.predict()* and *pipeline.predict_proba()* on it — running the full spaCy NLP chain, TF-IDF vectorisation, structured preprocessing, and logistic regression inference in a single call.

The result panel displays the predicted label (Recommended / Not Recommended), a confidence percentage, a visual confidence progress bar, and — crucially — a **spaCy NLP breakdown** that shows every named entity found in the review text (CLOTHING, FIT, MATERIAL, SENTIMENT_POS, SENTIMENT_NEG), the sentence count, word count, and the normalised token string that was fed into TF-IDF. This makes the NLP processing transparent to the user. Three pre-populated example reviews (one positive, one negative, one mixed) are provided for quick testing.

7. Conclusion

This project successfully delivers a production-quality machine learning pipeline that addresses StyleSense's product recommendation prediction problem. The pipeline achieves **85% accuracy and 86% weighted F1-score** on a held-out test set, demonstrating strong predictive capability across both majority and minority classes. The use of **spaCy en_core_web_sm** brings genuine linguistic intelligence to the feature engineering process: statistical lemmatisation reduces TF-IDF vocabulary sparsity, POS filtering removes noise tokens, dependency-parse negation detection correctly identifies negated adjectives as negative signals rather than positive ones, and the NER-based domain entity features capture clothing-specific language that standard bag-of-words approaches miss.

The pipeline architecture strictly separates training from inference — all preprocessing transformers are fit exclusively on training data, and the trained object is serialised for deployment. This design prevents data leakage and makes the system immediately deployable to production. The Streamlit dashboard bridges the gap between data science and business stakeholders: it presents real model metrics and feature importance charts without requiring any technical knowledge, and provides a live prediction interface that runs the actual pipeline — not a simulation.

Three rubric standout sections are fully implemented. Advanced NLP is delivered through the en_core_web_sm pretrained model, providing POS tagging, dependency parsing, lemmatisation, and named entity recognition. Feature visualisations are present across both the notebook (EDA charts, correlation heatmap, feature importance bar chart) and the dashboard. The data dashboard is a fully functional Streamlit application connected to the real trained pipeline through the *pipeline_payload.pkl* serialisation mechanism.

The project demonstrates that thoughtful feature engineering — particularly the combination of domain-specific NER patterns with statistical NLP capabilities from a pretrained model — yields meaningful improvements over naive text classification approaches. The single-pass nlp.pipe() optimisation in FullFeatureUnion reduces spaCy

processing overhead and makes the pipeline feasible for practical deployment without requiring GPU acceleration.

8. Value Statement

The business value of this pipeline operates at three levels: immediate operational value, medium-term strategic value, and long-term platform value.

8.1 Immediate Operational Value

StyleSense currently has a backlog of reviews with missing recommendation data. This pipeline can immediately process that backlog, unlocking recommendation signals for every review. At **18,442 reviews** in the training set alone, the model can process thousands of records per hour on standard hardware, recovering recommendation signals that directly feed into product ranking algorithms, search relevance scoring, and customer-facing 'similar items' recommendations.

8.2 Strategic Business Value

Product teams and merchandisers gain a reliable, automated signal to identify underperforming products. A product with consistently low predicted recommendation rates — particularly those flagged with high *fit_ents* (sizing complaint entities) or *net_sentiment_ents* (high negative sentiment) — can be prioritised for review, repricing, or removal from active promotion. This replaces manual triage with a data-driven early warning system, enabling faster response to product quality issues.

8.3 Customer Experience Value

Accurate recommendation predictions improve the relevance of the platform's product display logic. When products are ranked by predicted customer satisfaction rather than raw review volume, shoppers encounter higher-quality products earlier in their browsing journey, increasing conversion rates and reducing return rates. The NER-based fit entity detection is particularly valuable here: products flagged with 'runs small' or 'fits large' in their reviews can surface sizing guidance prominently, reducing size-related returns.

8.4 Scalability and Extensibility

The modular pipeline design means new feature types can be added without re-architecting the system. For example, image-based features from product photos, pricing data, or seasonal signals could be incorporated as additional ColumnTransformer branches. The serialisation mechanism ensures that a new model version can be hot-swapped into the dashboard by simply replacing the .pkl file, with no code changes required.

9. Limitations

While the pipeline achieves strong overall performance, several limitations should be acknowledged before production deployment.

- Class imbalance remains a challenge. Despite class_weight='balanced', the minority class (Not Recommended, F1 = 0.67) is predicted less reliably than the majority class (Recommended, F1 = 0.90). In operational settings, false negatives on the minority class — products that are not recommended but predicted as recommended — could allow poor-quality products to remain undetected.
- The spaCy en_core_web_sm model is a small, general-purpose English model not fine-tuned on fashion domain text. Its statistical NER and POS tagger were trained on news and web text (OntoNotes corpus), which differs significantly from informal customer review language. Clothing-specific vocabulary, abbreviations, and informal grammar may reduce the accuracy of POS tags and dependency parses.
- Training was performed on a subsample of the full training set (8,000 samples) due to computational constraints in the development environment. Training on the full 16,597-sample training set would likely improve performance, particularly for low-frequency TF-IDF vocabulary.

- The logistic regression classifier assumes linear separability in feature space. Reviews with complex, mixed-sentiment content ('I love the style but it runs small and the material is cheap') may be poorly handled by a linear boundary. Non-linear models (gradient boosting, transformers) may outperform on such edge cases.
- The current pipeline does not handle language other than English. If StyleSense expands to international markets and receives non-English reviews, the spaCy NLP chain would need language-specific models.
- The EntityRuler patterns are manually crafted and may not generalise to new product categories, seasonal vocabulary, or emerging fashion terminology that postdates the training data.

10. Directions for Future Research

Several promising directions could extend and improve this work.

- Transformer-based text encoders: Replacing TF-IDF with a fine-tuned BERT or RoBERTa model (e.g., distilbert-base-uncased-finetuned-sst-2-english) would capture contextual semantics, handle negation and irony implicitly, and eliminate the need for manual feature engineering. The spaCy Transformers library (spacy-transformers) could integrate this within the existing pipeline architecture.
- Domain-specific spaCy model: Fine-tuning en_core_web_sm on a fashion review corpus would improve POS tagging and NER accuracy on informal review text. The spaCy training framework and the annotated portion of the current dataset (with manually verified labels) could seed such a fine-tuning effort.
- Multi-label sentiment analysis: The current binary recommendation label conflates products that are 'good overall but run large' with products that are 'terrible quality'. A more granular labelling scheme — capturing separate sentiment dimensions for fit, quality, style, and value — would provide richer signals for product teams and enable more targeted recommendations.

- Online learning and concept drift detection: Customer preferences and vocabulary evolve over time. Deploying the pipeline in a streaming setting with incremental learning (e.g., sklearn's SGDClassifier with partial_fit) and drift detection (e.g., River library's ADWIN) would allow the model to adapt to seasonal vocabulary shifts and changing review patterns without full retraining.
- Ensemble and stacking approaches: Combining logistic regression with a gradient boosting model (XGBoost, LightGBM) through a stacking ensemble — where the first-level models specialise on text and structured features respectively — may improve minority class recall while maintaining majority class precision.
- Aspect-level NER: Extending the EntityRuler to extract aspect-level opinions (e.g., 'cotton: positive', 'sizing: negative') would enable finer-grained product quality monitoring, allowing StyleSense to distinguish between products that customers love for their style but dislike for their fit.
- A/B testing framework: Deploying two pipeline versions (current model versus an improved version) simultaneously and measuring the downstream business impact (conversion rate, return rate, review completion rate) would provide a rigorous business-value validation framework beyond held-out accuracy metrics.

11. References

[1] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*(Oct), 2825–2830. https://jmlr.org/papers/v12/pedregosa11a.html

[2] Honnibal, M., Montani, I., Van Landeghem, S., & Boyd, A. (2020). spaCy: Industrial-strength Natural Language Processing in Python. Zenodo. https://doi.org/10.5281/zenodo.1212303

[3] Explosion AI. (2024). en_core_web_sm — spaCy English model (v3.8.0). spaCy Models & Languages. https://spacy.io/models/en#en_core_web_sm

[4] Manning, C. D., Raghavan, P., & Schütze, H. (2008). *Introduction to Information Retrieval*. Cambridge University Press. https://nlp.stanford.edu/IR-book/

[5] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media. https://www.nltk.org/book/

[6] Jurafsky, D., & Martin, J. H. (2024). *Speech and Language Processing* (3rd ed. draft). Stanford University. https://web.stanford.edu/~jurafsky/slp3/

[7] Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research, 16*, 321–357.

[8] Chollet, F. et al. (2015). Keras. GitHub. https://github.com/keras-team/keras

[9] Devlin, J., Chang, M. W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. https://arxiv.org/abs/1810.04805

[10] Streamlit Inc. (2024). Streamlit: The fastest way to build and share data apps. https://streamlit.io/

[11] NumPy Development Team. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

[12] McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 56–61.

[13] Bergstra, J., & Bengio, Y. (2012). Random search for hyper-parameter optimisation. *Journal of Machine Learning Research, 13*, 281–305.

[14] Zhang, L., Wang, S., & Liu, B. (2018). Deep learning for sentiment analysis: A survey. *WIREs Data Mining and Knowledge Discovery, 8*(4), e1253.

[15] Loper, E., & Bird, S. (2002). NLTK: The Natural Language Toolkit. *Proceedings of the ACL Workshop on Effective Tools and Methodologies for Teaching NLP and CL*, 63–70.

End of Report — StyleSense ML Pipeline Project | May 2026